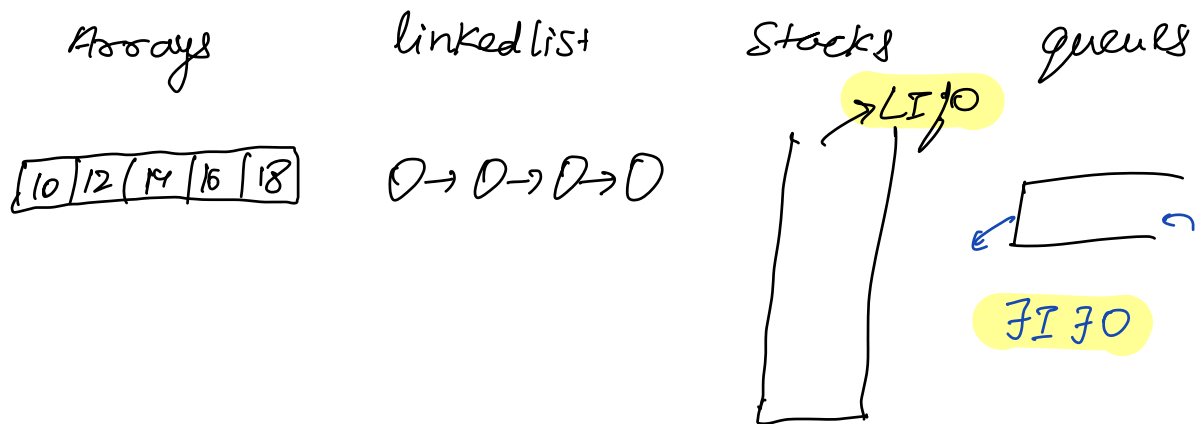


- Subhesh
- DTU
- 5th on codechef
- PerCoding → 3 yrs of teaching

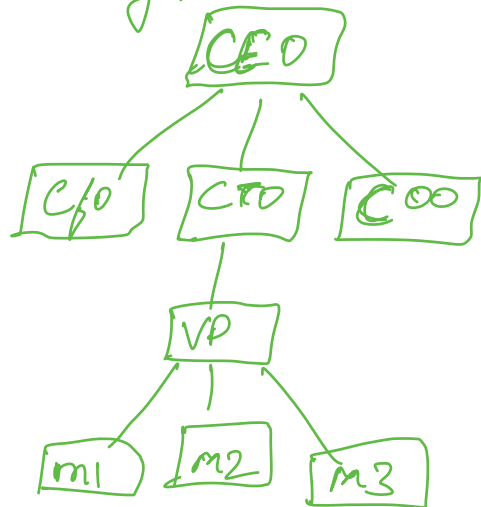
Today's Agenda: Trees



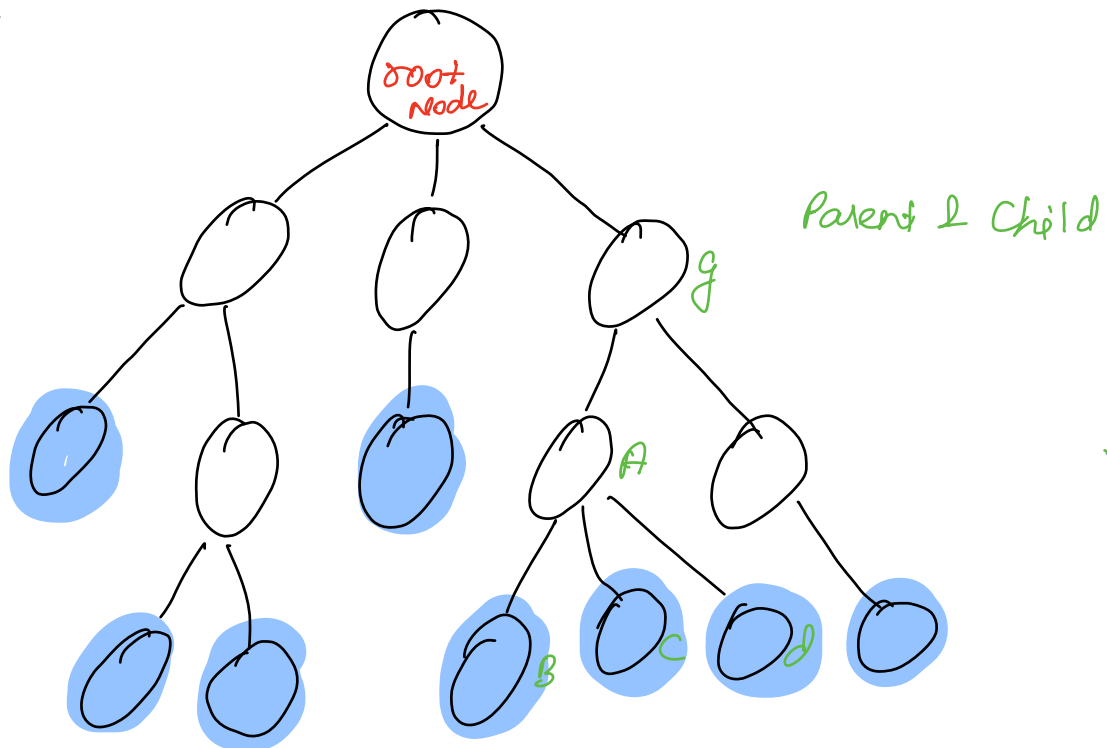
Hierarchical Data :

- (i) family tree
- (ii) Company Org Structure
- (iii) file system
- (iv) git

(i) Company :



Trees :

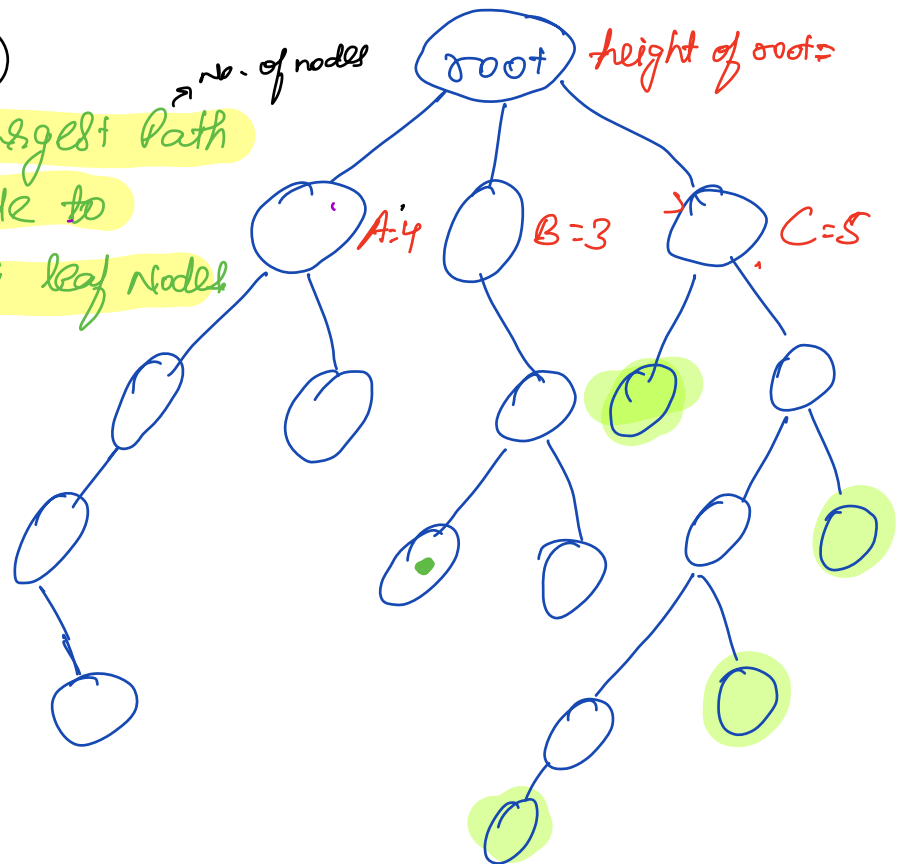


Naming Conventions :

- (i) Node :
- (ii) root node : The node from where tree is originating.
- (iii) edge : Line connecting two nodes.
- (iv) Parent :
- (v) Child :
- (vi) Sibling : Nodes having same parent.
- (vii) leaf nodes : Nodes without any children.
- (viii) Ancestor : all nodes from node to root node are its ancestors
- (ix) Descendant :

* Height (node)

↳ length of largest path
from that node to
any of descendant leaf nodes



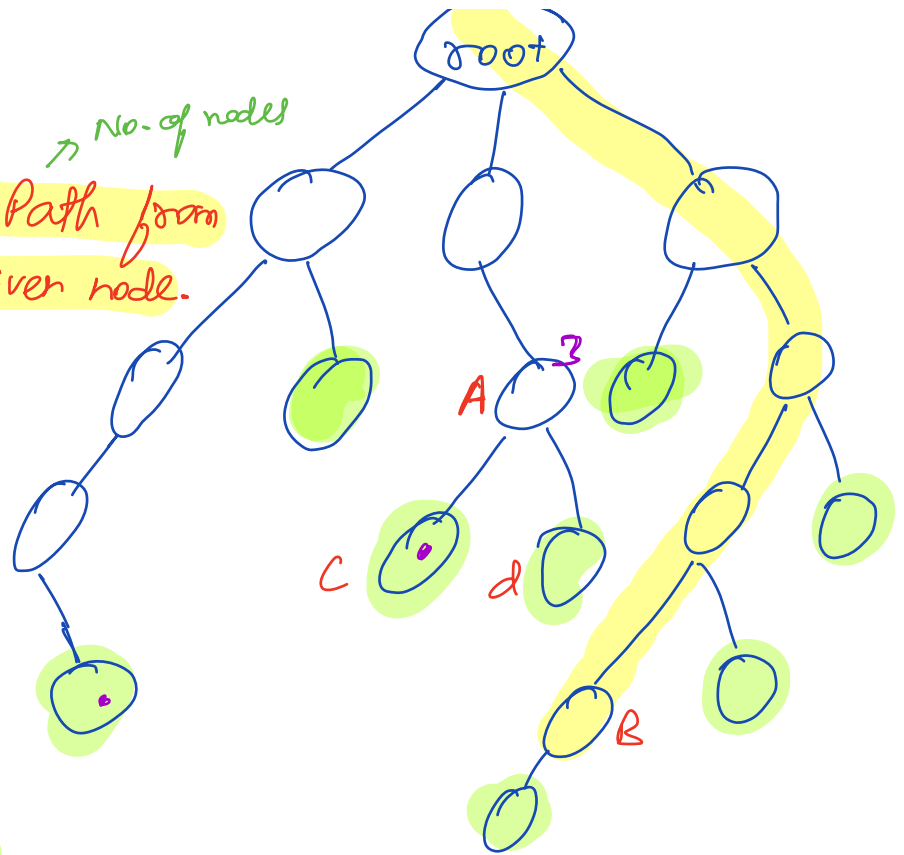
Obs: Height (node) = max of (height of all child nodes) + 1.

Note: Height (leaf nodes) = 1

* DEPTH

↳ length of Path from root Node to given node.

→ No. of nodes



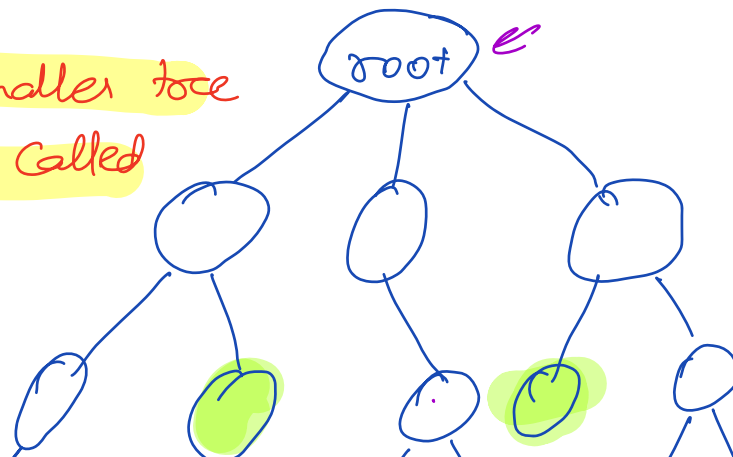
depth(A) = 3

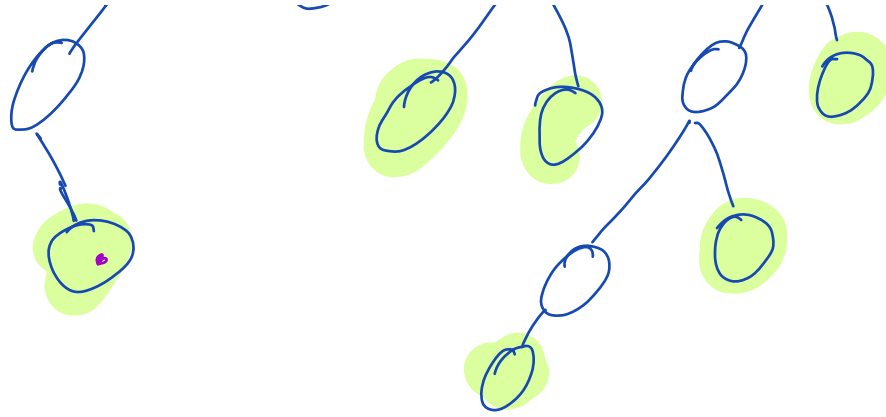
depth(B) = 5

obs: If $\text{depth}(\text{node}) = K$, then depth of all its child nodes will be $K+1$

* Subtree:

↳ Any smaller tree in a tree is called subtree.





* Types of tree :

(i) Binary tree :

Every node can at max have 2 child nodes.

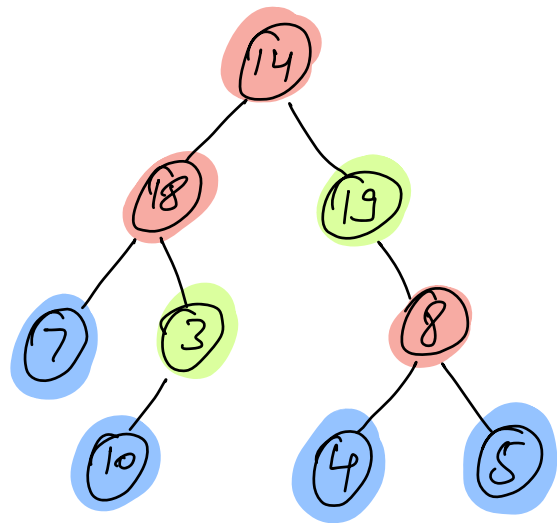
→ adv. module

(ii) N-ary tree :

Every node can at max have N child nodes.

* Binary tree

no children:  → 4
 1 child :  → 2
 2 child nodes:  → 3



```

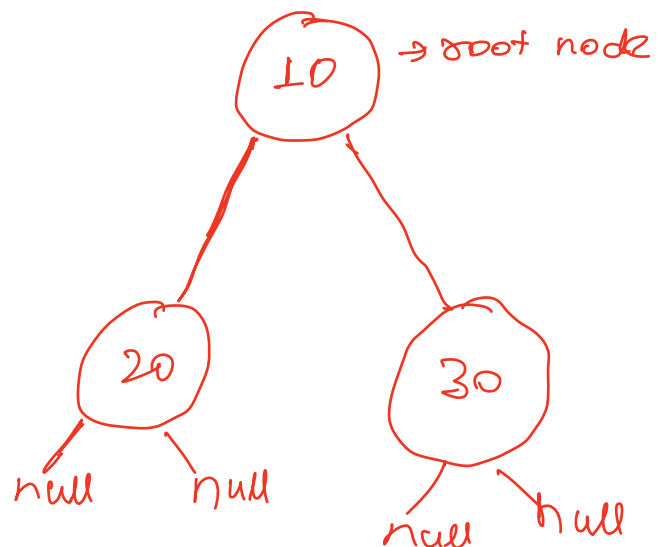
class Node {
    int data;
    Node left;
    Node right;
  
```

```

  Node(int n) {
    data = n;
    left = null;
    right = null;
  }
  
```

}

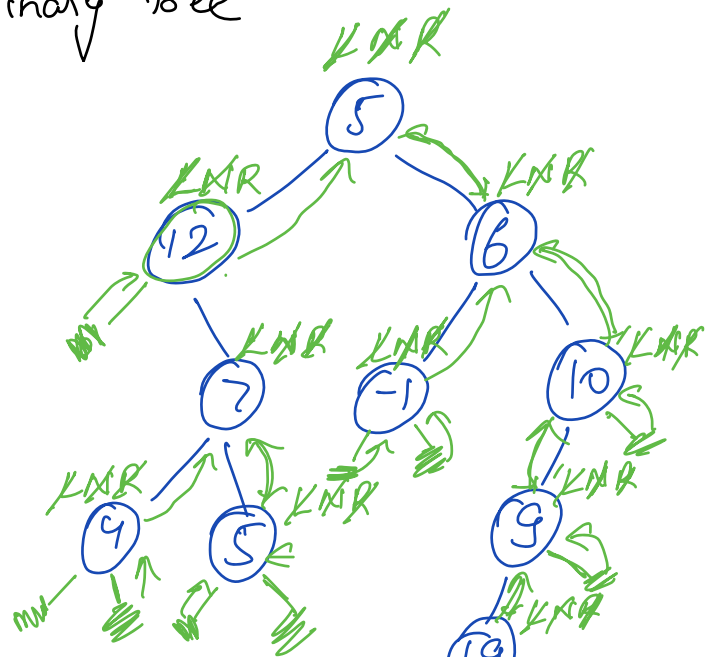
- (i) Node root = new Node(10)
- (ii) root.left = new Node(20)
- (iii) root.right = new Node(30)



Problems:

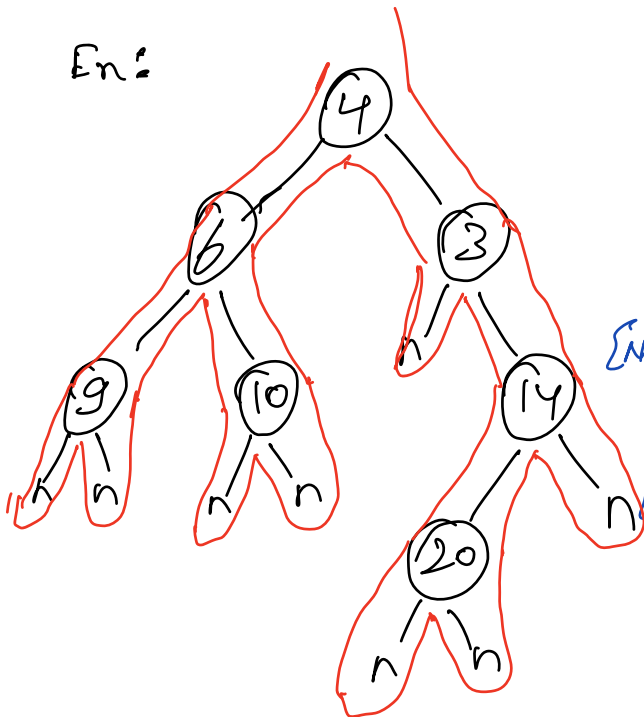
① Traversal in a binary tree

- ↳ Preorder: NLR
- ↳ Inorder: LNR
- ↳ Postorder: LRN



Inorder traversal: 12 4 7 5 5 -1 6 19 9 10

Ex:



Inorder: 9 6 10 4 3 20 14

[NLR] Preorder: 4 6 9 10 3 14 20

[LRN] Postorder: 9 10 6 20 14 3 4

// Inorder traversal → recursion

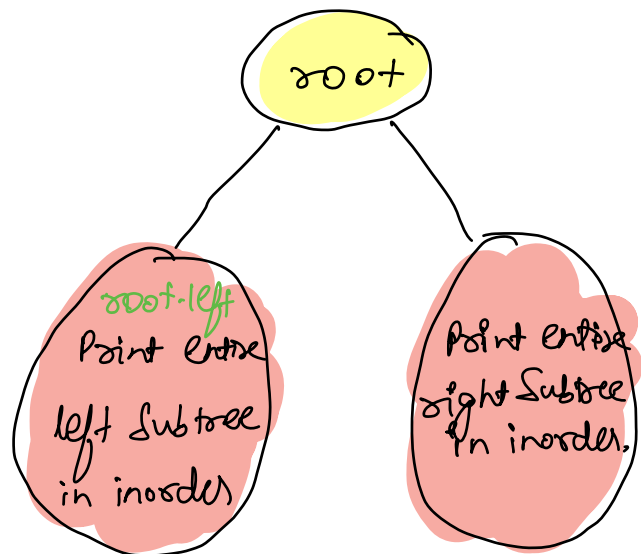
Recursion:

- ① **ask:** {decide, what your function should do, assume it does}
- ② **Main logic:** {solving problems with subproblems?}
- ③ **base case:** {when code stops}

[LNR]
* void **inorder**(Node root){
 ① if (root == null) return;
 ② **inorder(root->left);**
 ③ **Print (root->data);**
 ④ **inorder (root->right);**
}

ASS: Given root node,
it will print tree in
inorder.

main logic:



[LNR]
Inorder: 1 2 3 4

[NLR]
Preorder: 1 3 2 4

[LRN]
Postorder: 1 2 4 3

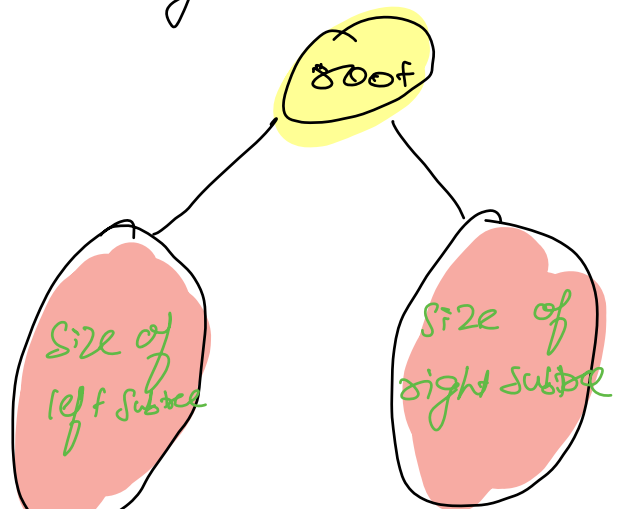
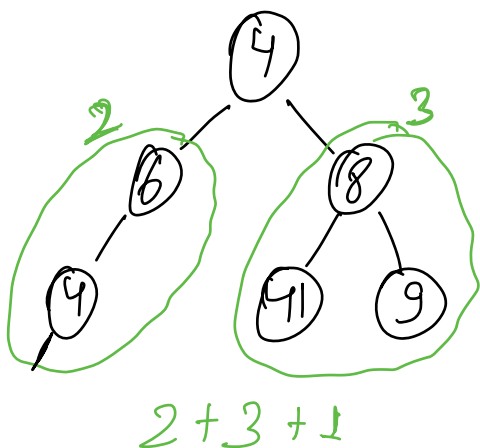
base case? $\text{if } (\text{root} == \text{null}) \{$
 return
 $\}$

* Prob: Size of the binary tree.
 ↳ No. of nodes in the binary tree

```
int Size (Node root) {
    if (root == null) {
        return 0;
    }
    int l = Size (root.left);
    int r = Size (root.right);
    return l + r + 1;
}
```

Ass: Given root node, it
 will return the size of
 binary tree.

main logic:



↳ $\text{Size} = \text{Size from left} + \text{Size from right} + 1$

base case if (root == null)
return 0

* Sum of all nodes:

```
int Sum(Node root) {  
    if (root == null) {  
        return 0;  
    }  
    int lsum = Sum(root.left);  
    int rsum = Sum(root.right);  
  
    return lsum + rsum + root.data;  
}
```

H.W: Code for height of the tree.

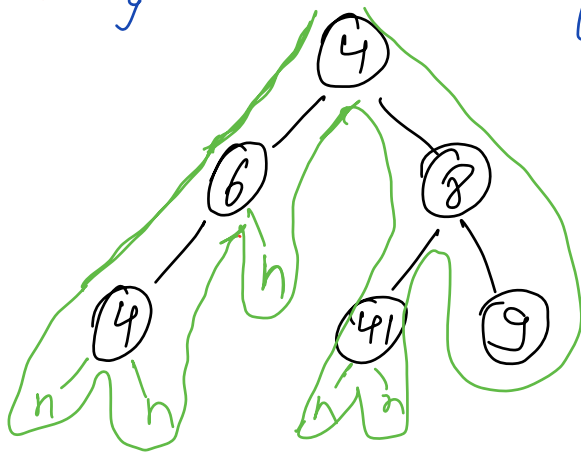
Size(A)
Size(A)

Trace

```

int Size (Node root) {
    ① if (root == null) {
        return 0;
    }
    ② int l = Size (root.left);
    ③ int r = Size (root.right);
    ④ return l + r + 1;
}

```



$O(2n) \approx O(n)$

Size(9)	l=0	r=0	1 2 3
Size(n)			1
Size(n)			1
Size(6)	l=0	r=0	1 2 3
Size(8)	l=1	r=1	1 2 3 4
Size(n)			1
Size(n)			1
Size(n)			1
Size(4)	l=0	r=0	1 2 3 4
Size(6)	l=1	r=0	1 2 3 4
Size(4)	l=2	r=3	1 2 3 4

$2 + 3 + 1 \Rightarrow 6$